

Computationally Efficient DNN Mapping Search Heuristic using Deep Reinforcement Learning

SUYASH BAKSHI and LENNART JOHNSSON, University of Houston, Main Campus, USA

In this work, we present a computationally efficient Reinforcement Learning mapping search heuristic for finding high quality mappings for N-dimensional convolution loops that uses a computationally inexpensive reward function based on potential data reuse of operands to guide the search process. We also present a RL state representation generalizable to N-dimensional convolution loops, and a state representation parsing strategy ensuring that only valid mappings are evaluated for quality. Our RL search heuristic is applicable to multi-core systems with a memory hierarchy. We show that our RL based search heuristic for a range of 3D convolution layers, at significantly lower computational expense than random search, generally yields mappings with lower Energy-Delay Product (EDP) for an architecture with multiple processing elements with shared memory connected to DRAM. Our evaluation results demonstrated across 19 3D convolution layers, shows that our RL method performed only an average 11.24% of the operations of that of Timeloop's random search for assessing same number of valid mappings. The mappings found using Timeloop had an average 12.51% higher EDP compared to lowest EDP mapping found using our RL method. Further, the lowest EDP mappings found using our method had an average only 4.69× higher EDP than the theoretical lower bound EDP, with the best case being only 1.29× higher.

CCS Concepts: • **Computer systems organization → Single instruction, multiple data**; • **Computing methodologies → Reinforcement learning;**

Additional Key Words and Phrases: Reinforcement learning, DNN, mapping search, convolution

ACM Reference format:
Suyash Bakshi and Lennart Johnsson. 2023. Computationally Efficient DNN Mapping Search Heuristic using Deep Reinforcement Learning. *ACM Trans. Embedd. Comput. Syst.* 22, 5s, Article 115 (September 2023), 21 pages.
https://doi.org/10.1145/3609110

1 INTRODUCTION

Achieving high performance and energy efficiency of applications on hardware architectures require significant tuning and optimization efforts [4, 9, 26, 28, 34]. For CPUs and GPUs, vendors generally provide highly optimized libraries implemented by domain experts e.g., MKL [6], CuBLAS [8]. But highly efficient implementations require adaptivity to problem specific parameters such as size and dimensionality of the problem at hand, a level of adaptivity not common in software

This article appears as part of the ESWEEK-TECS special issue and was presented in the International Conference on Compilers, Architectures, and Synthesis for Embedded Systems (CASES), 2023.
Authors' address: S. Bakshi and L. Johnsson, Department of Computer Science, Philip Guthrie Hoffman Hall, 3551 Cullen Blvd, Room 501, Houston, Texas, 77204-3010; emails: sbakshi@uh.edu, johnsson@cs.uh.edu.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
1539-9087/2023/09-ART115 $15.00
https://doi.org/10.1145/3609110

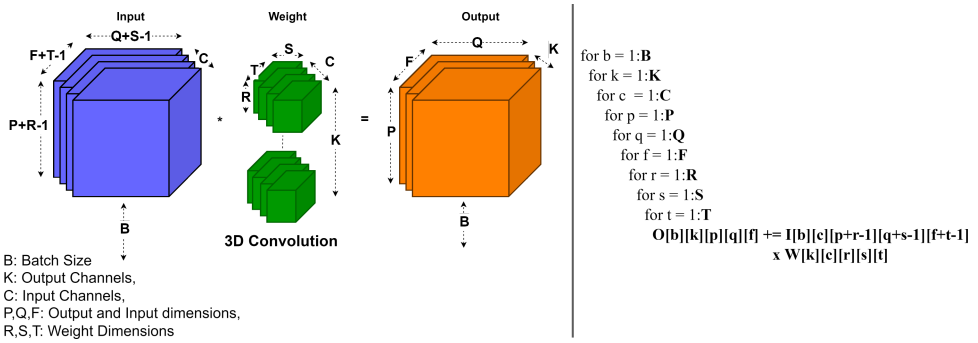


Fig. 1. A 3D convolution operation (N = 3): (left) illustration, (right) loop nest.

libraries. Further, the drive for increased energy efficiency in computation has resulted in the emergence of Domain Specific Architectures (DSAs) [16]. For Deep Neural Networks (DNNs) examples thereof are [11, 24], and several research prototypes, such as [14, 19, 37]. Hand-tuning for each architecture is an arduous task. Thus, automatically mapping the execution of DNNs (i.e., determining loop tiling, loop ordering and parallelization strategy) onto hardware architectures yielding high performance and energy efficiency is a key challenge.

DNNs [27, 39, 40, 47] comprise of a collection of layers performing convolutions, pooling, batch normalization, and full interconnects, with the majority of operations performed in the convolution layers. The DNN input data dimensionality is application dependent, and each element may have several attributes typically referred to as channels. Hence, for N-dimensional data the input data set may be N+1 dimensional. Depending on the application the dimensionality of the convolution kernel may range from "zero" (point-wise) to N. Convolution kernel elements are typically referred to as weights. Different convolution kernels may be employed to extract different features of the input data resulting in separate Output data sets for each feature extracted, known as Output channels. Hence, the dimensionality of the output data set generally is also N+1 and convolution layers having N+2 nested loops. The input data is often partitioned into batches adding one additional loop for the complete processing of the input data. Figure 1 shows a convolution layer for 3D data with each data element having several attributes (C channels) with 3D convolution kernels (Weights) or the extraction of several features (K output channels). There are three loops for the dimensions of the convolution kernel (R,S,T), three loops for the dimensions of the input data set (P,Q,F), one loop for the Input channels (C) and one loop for the Output channels (K) and finally one loop covering batches (B).

A resource and energy efficient convolution layer mapping depends on its input and output parameters and hence typically is unique for each layer and application. A common approach to achieve low execution times and minimize energy consumption is to schedule operations to maximize data reuse for each level of a memory hierarchy [15, 23, 32]. High data reuse is achieved through partitioning of loops to implicitly block (tile) arrays according to the memory hierarchy and the scheduling of loops. Tile sizes are fitted for each level of the memory hierarchy and the number of interacting operands that should be co-located at the memory level to minimize overall data movement. For e.g., 3D Convolution Neural Networks (CNNs) (used for evaluation in this work) the loop ranges typically allow for many possible tilings. For example, in processing seismic data volumes of $650 \times 950 \times 462$ voxels [5] a 3D U-Net layer may have 32 input and 64 output channels and $3 \times 3 \times 3$ convolution kernels (weights). To reduce the search space for performance and energy efficient tilings and their mapping many approaches restrict tile sizes to be prime factors of the loop ranges [22, 31, 36] (referred as "loop range prime factors" or "LPFs" in our

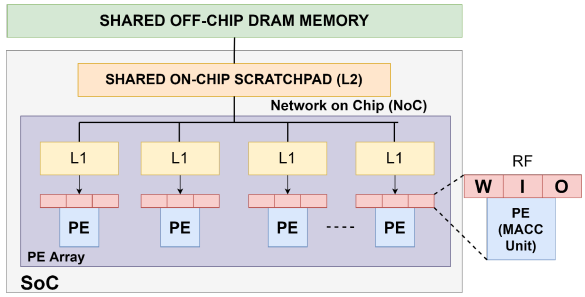


Fig. 2. Hardware architecture model with a PE array, with each PE containing a memory hierarchy (L0 and L1), a shared on-chip scratchpad memory (L2) and off-chip DRAM memory.

work). But even with this restriction the number of possible tilings is extremely large, 26! = 4.03 $\times 10^{25}$ for the seismic processing example (total 26 LPFs in the example). In this work we use LPF based tile sizes and operand reuse as a guide to find efficient mappings.

We propose a computationally efficient Reinforcement Learning (RL) search heuristic for finding high quality convolution layer mappings for N-dimensional data [27, 39, 40, 47] on to multi-PE hardware architectures with a memory hierarchy (Figure 2) and shared memory levels. We utilize the RL training process as the search heuristic to locate high quality mappings thereby eliminating the need for training data. In our RL approach mappings are referred to as states. The RL Agent determines state transitions to be carried out by the Environment that gives the Agent a reward based on the potential operand reuse in mappings. It is considerably simpler to evaluate than XGBoost [12] used for generating reward in [3]. Our main contributions are:

(1) A novel RL reward function based on potential data reuse of the operands, computationally inexpensive compared to determining actual reuse or execution time [3].
(2) A mapping pruning strategy based on potential operand reuse.
(3) A novel linearized mapping state representation for n-deep nested loops and parallel hardware architectures with an m-level memory hierarchy.
(4) A state representation parsing strategy ensuring that only valid mappings are generated.

We compare our method against Timeloop's [36] random search heuristic state-of-the-art mapper in regard to (1) search computational requirements and (2) Energy-Delay-Product (EDP) of mappings found.

2 STATE-OF-THE-ART MAPPERS

Several mappers (Table 1) have been proposed including the use of exhaustive search with pruning mechanisms [17, 29, 31, 36, 44, 45]. The quality of mappings is typically assessed using analytical models with parameters for operation times and energies. Other works have utilized RL [3, 46], Genetic algorithms (GA) [25], Simulated Annealing (SA) [13], Constrained Optimization [22], or supervised learning with gradient based search [20]. Mind Mapping's [20] used training datasets of 1M or more for one hardware architecture. Methods requiring training data sets have the drawback that acquiring, or generating, potentially extensive data sets can be both difficult and costly, and the applicability of training outcome to architectures and cost models not captured in the training data may be limited. Methods producing mappings by gradient updates need to employ techniques like Projected Gradient Descent [33] to ensure that mappings are valid (e.g., tile sizes). The RL reward function in Chameleon [3] uses estimates of mapping performance produced by a machine learning model [12], and in FlexTensor [46] by actual hardware measurements. A reward function

Table 1. State-of-the-art DNN Mapping Search Frameworks

Framework	Search Algorithm	Mapping Cost Measurement/Estimation	Architectural Model Supported
Timeloop [36]	Brute-force pruned and Random search	Analytical model	PE array with multi-level memory hierarchy
ZigZag [31]	Brute-force pruned search	Analytical model	
dMazeRunner [17]	Brute-force pruned search	Analytical model	
Intel Stripe IR [45]	Brute-force pruned search	Actual Hardware Measurements	
Interstellar [44]	Brute-force pruned search	Analytical model	
MAESTRO [29]	Tiling factors search for 6 predefined mapping templates	Analytical model	
TVM [13]	Simulated Annealing	Gradient Boosted Trees Trained using Actual Hardware Measurements	
Chameleon [3]	Reinforcement Learning using mapping performance as the reward		
Mind Mappings [20]	Supervised Learning (Gradient based search)	Multi-layer Perceptron Trained using Mapping cost datasets collected via Analytical cost model	
FlexTensor [46]	Brute-force pruned search + Reinforcement Learning	Actual Hardware Measurements	
GAMMA [25]	Genetic Algorithm	Analytical model	
CoSA [22]	Constrained Optimization	Analytical model	
Ours	Reinforcement Learning using potential data reuse as reward	Analytical model	

Table 2. Loop Relevance (r), Irrelevance (ir) and Partial Relevance (pr) Relationships of Operands for 3D Convolution shown in Figure 1

Loops	R	S	T	P	Q	F	C	K	B
Loop IDs	1	2	3	4	5	6	7	8	9
Weight	r	r	r	ir	ir	ir	r	r	ir
Output	ir	ir	ir	r	r	r	ir	r	r
Input	pr	pr	pr	pr	pr	pr	r	ir	r

based on mapping performance may not capture the differences in dynamic energy consumption of mappings as mappings with similar runtimes can have different dynamic energy consumption based on the amount of data moved between memory levels of the hardware (as shown in [9] (Table 2)).

3 ARCHITECTURE MODEL

We designed our mapping search method to support architectures with multiple processing elements (PE arrays) with their own memory hierarchy and a shared memory. Conceptually, the processing elements and the shared memory are considered as a System on Chip (SoC). The shared memory is connected to an external "global" memory, DRAM. In this work we assume a communication infrastructure that enables inter PE and PE to shared memory communication without contention other than limitations due to individual PE and shared memory port capacities. PEs each have one MAC (multiply-accumulate) unit, a Register File (RF) for MAC unit input/output, a local memory hierarchy, and a network interface. Figure 2 shows the basic elements of the model architecture with one local memory level (L1). The essential features are similar to those of DNN accelerators such as [14, 19, 24].

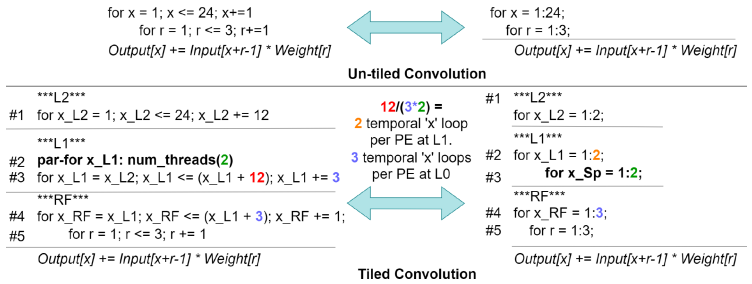


Fig. 3. (Top) Un-tiled convolution loop. On the left in C syntax and on the right in loop prime factor syntax. (Bottom) Tiled convolution loop for an example 2 PE architecture, each with 2 levels of private memory hierarchy (L0 and L1) and a shared L2 memory.

4 MAPPING COST ESTIMATION MODEL

The quality of a mapping is measured by execution time, energy, and the energy-delay product (EDP). In assessing the execution time we assume each of the following operations require one cycle: MAC instructions, data movement between (1) adjacent memory levels, (2) PEs and (3) PE to shared memory. Energy estimates are based on energy estimates for a MAC instruction and for memory energy per read and write operation at each level of the memory hierarchy. We use 28nm CMOS technology energy values from [44] and CACTI 7.0 [10] for the results in Section 7. We use the Timeloop [36] framework to model the hardware architecture, execution time, and energy characteristics.

5 REPRESENTATION OF TILED LOOP NEST MAPPINGS TO MULTI-PE SYSTEMS WITH A MEMORY HIERARCHY

We use LPFs as the basis for tiling. Figure 3(top) shows an un-tiled 1D convolution operation with a 3-element (Weight) kernel, on the left in a typical C language syntax and on the right using the LPFs. In the example a 26 element input operand (indexed by 'x+r-1') convolved with the 3 element Weight operand (indexed by 'r') generates a 24 element Output operand (indexed by 'x'). Figure 3(bottom) shows the convolution loop tiled for a hardware architecture with two PEs. The example assume PEs with a Register File (RF) and one level of local memory (L1) and a shared L2 memory (as in Figure 2). The loop 'r' is un-tiled, and the 'x' loop is tiled at L2 with a tiling size of 12 (x_L2 at line #1). This results in a total tile size of 12 being brought into L1. Further, the x_L1 loop with tile size of 12 is parallelized across the two PEs using the 'par-for' (shown as x_Sp (spatial) at line #3 in LPF syntax), resulting in tiles of size 6 each being distributed across the private L1 memory of the PEs. Finally, the tile of size 6 for each PE is further tiled to read a sub-tile of size 3 into the Register File (RF) of each PE. The loops x_L2, x_L1, x_RF and r determine the tile sizes and data movement between the L2, L1 and RF memories, thus define the temporal scheduling of computations. The loop x_Sp determines the parallelization strategy and mapping of concurrent tasks to PEs and define the spatial scheduling of computations. In our implementation for 3D data sets, Input and Output operands, and convolution kernels are represented as dense 5-D array data structures.

6 RL BASED MAPPING SEARCH HEURISTIC

6.1 State Representation

Convolution layer loops are assigned a loop ID, and LPFs are computed for all loops. The LPFs together with their corresponding loop IDs are stored as a 2-tuple (ID, LPF) in a tuples-list. There

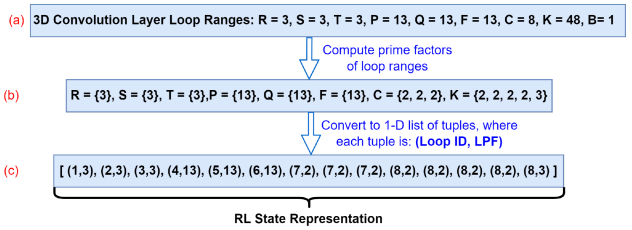


Fig. 4. RL state representation for a 3D convolution layer.

is no imposed ordering of 2-tuples in the list. The number of list items equals the total number of LPFs in the loop nest. The list encodes no information about the hardware architecture. However, the order of the 2-tuples does matter for the mapping, that processes the list in a left-to-right order in assigning loop iterations to PEs (spatial scheduling) and for each PE operand tiles to its memory hierarchy (temporal scheduling) as illustrated in Section 5 and described in detail in Section 6.4.2. Reordering of the tuples in the list to achieve high quality mappings are decided by the agent based on rewards from the environment. The tuples-list together with the outcome of the mapping process result in a mapping of tiles and their Loop IDs to the hardware architecture, a State in context of the RL search method. Since our proposed State representation depends only on the number of LPFs for a convolution layer, it is hence agnostic to convolution dimensionality and can be applied to N-dimensional convolutions.

Using the 3D convolution loop nest in Figure 1 for illustration with loop ranges of R, S, T = 3, P, Q, F = 13, C = 8, K = 48, and B = 1, all but the C and K loop ranges are prime factors, LPFs. For the C loop the LPFs are (2,2,2) and for the K loop they are (2,2,2,2,3) as shown in Figure 4(b). With loop IDs as follows R': 1, 'S': 2, 'T': 3, 'P': 4, 'Q': 5, 'F': 6, 'C': 7, 'K': 8, 'B': 9, Figure 4(c) shows a corresponding 2-Tuples list for a batch (B loop omitted). The 2-tuples are order by loop ID and for each loop ID in order of non-decreasing LPFs for illustration clarity.

6.2 RL Method Overview

The two major components in our RL mapping search heuristic are: (1) the RL agent and (2) the environment as shown in Figure 5. The agent determines 2-tuple reorderings, actions, starting with a user specified tuples list and a randomly initialized policy. For every tuple-list reordering, the agent receives a reward from the environment. The agent's goal is to determine tuple reorderings leading to the highest rewarding mappings. Actor-Critic neural networks representing the policy are trained using the gathered information of tuple-lists and their associated rewards for the agent to make favorable list reordering determinations. Our Actor-Critic networks use the Proximal Policy Optimization (PPO) [38] learning algorithm. The environment carries out the agent determined actions. The environment then computes the potential reuse for each operand for the mapping, which is fed to the reward function to compute a reward. The agent then stores this "tuple-list – action – reward" transition.

The search process is carried out as $N_{episodes}$ episodes each of "total trajectory length", $N_{totalTrajectory}$ "tuple-list – action – reward" transitions. Within each episode training of the Actor-Critic neural network is based on distinct sets of "learning trajectory length", $N_{learningTrajectory}$ "tuple-list -action-reward" transitions. Thus, within each episode $N_{totalTrajectory}/N_{learningTrajectory}$ training events on distinct data are performed. For each training event, the $N_{learningTrajectory}$ "tuple-list -action-reward" transitions are divided into batches of $N_{batchSize}$ transitions. Each batch is used to perform N_{epochs} epochs of training of the Actor-Critic neural networks, involving the execution of the PPO learning algorithm, backpropagation,

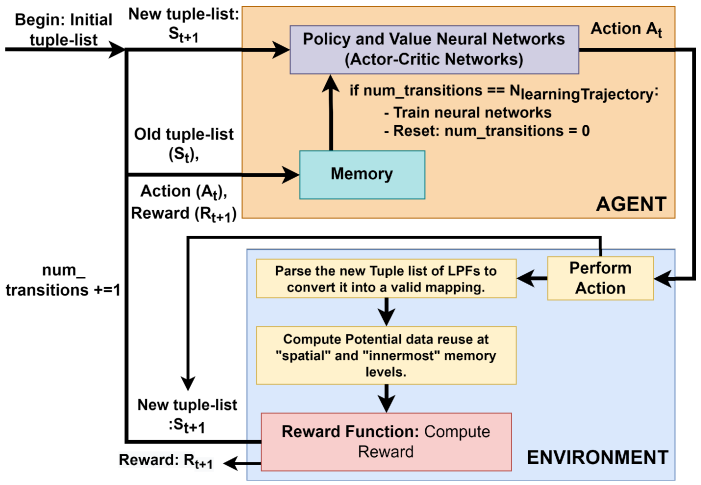


Fig. 5. RL based mapping search heuristic overview.

and neural network weights updates. The total number of training iterations for the entire search process is given by Equation (1) below:

$$N_{TrainingIterations} = (N_{totalTrajectory}/N_{learningTrajectory}) * (N_{learningTrajectory}/N_{batchSize}) * N_{epochs} * N_{episodes}$$

$$N_{TrainingIterations} = N_{totalTrajectory}/N_{batchSize} * N_{epochs} * N_{episodes} \quad (1)$$

For each training iteration, the PPO [38] learning algorithm requires an inference of the Actor-Critic networks for the batch of transitions. Further, computing gradients for the backpropagation requires an estimated 2× the operations per inference of the networks [21, 30]. The total operations for training are given by Equation (2):

$$Ops_{training} = N_{TrainingIterations} * ((2 + N_{batchSize}) * (OpsPerInference_{Actor} + OpsPerInference_{Critic})) \quad (2)$$

Once the neural networks are trained, the agent's memory is cleared to allow for storing the next set of reorderings. At the end of each episode, the agent is reset to the initial tuple-list. The implementation is described below.

6.3 Agent

The Actor neural network represents the policy and is responsible for action generation. The Critic (Value) network estimates the value of the Agent input tuple-list, i.e., it "criticizes" the Actor network's reordering that resulted in the input tuple-list. The Critic network's output is used during the training step to compute gradients for backpropagation, such that future actions generated by Actor are improved. The tuple-list is input to both networks. Each network in our implementation has three layers, two fully connected (FC) hidden layers each with 20 neurons and a ReLU activation function followed by a final FC layer. In the Critic network it has one neuron. In the Actor network it is composed of "N_{lpf}" fully connected sublayers, each sublayer with a set of three neurons and a softmax activation function for each set. N_{lpf}, is the number of tuple positions in

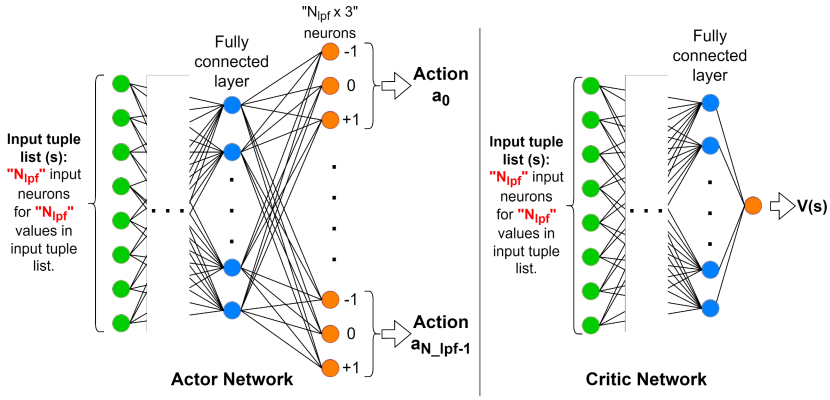


Fig. 6. Actor and critic neural networks.

the list. The three neurons for each list position represent the actions of swapping its 2-tuple with the 2-tuple in the preceding (−1) or succeeding (+1) list position or no action (0), Figure 6. The action to be taken for each list position is sampled from the probability distribution defined by the softmax activation function. The resulting list actions are processed in a left-to-right order. Hence, e.g., if in processing the action for a position its 2-tuple is swapped with the 2-tuple in the succeeding position, and when the action on the succeeding position is to swap its 2-tuple with that in the preceding position, the net effect of the two actions is no reordering. But, if the second action would have been no swap with neighbor list position, the net effect would be a swap of 2-tuples between the first and second positions. The actions result in a permutation of the tuple-list. The use of random sampling of actions for each list position makes our RL policy stochastic in nature that enhances the likelihood of finding high quality mappings.

The computational requirements for the Actor-Critic network are as follows. The number of operations for a FC layer is "(2 * n1 * n2)", where n1 is the number of inputs from the preceding layer and n2 is the number of neurons in the layer itself. The total operations per inference for our Actor and Critic networks are given by Equation (3) and (4):

$$OpsPerInference_{Actor} = (160 * N_{lpf} + 800) \tag{3}$$

$$OpsPerInference_{Critic} = (40 * N_{lpf} + 840) \tag{4}$$

The memory requirements are given by Equation (5) and 6:

$$MemoryReq_{Actor} = (80 * N_{lpf} + 400) * 4Bytes \tag{5}$$

$$MemoryReq_{Critic} = (20 * N_{lpf} + 420) * 4Bytes \tag{6}$$

We found that Actor-Critic networks with 256 neurons/layer did not result in any significant improvements in mapping quality. Moreover, to our knowledge, the Actor-Critic networks described above are similar to those of [3], but our method for determining actions is different.

6.4 Environment

The RL environment (Figure 5) operates in four steps: (1) perform the tuple-list permutations decided by the agent, (2) parse the new list and convert it into a valid mapping, (3) analyze the valid mapping's operand potential reuse using the loop relevance principle [31], and finally (4) generate a reward for the mapping given its operand potential reuse. To convert the tuple-list to a valid mapping hardware architecture information is used. The operand memory requirements

Table 3. Loop Relevance (r), Irrelevance (ir), and Partial Relevance (pr) for Convolution Operands along with Loop IDs for 0-N-dimensional Weights, N-dimensional Input and Output Feature Maps, with 1 Additional Dimension Each for Input Channels, Output Channels, and Batch Size

	Weight Dimensions (0-N-dim)	Input/Output Dimensions (N-dim)	Input Channels (1-Dim)	Output Channels (1-Dim)	Batch Size (1-Dim)
Loops IDs	[0, N]	[N+1, 2N]	2N+1	2N+2	2N+3
Weight	r	ir	r	r	ir
Output	ir	r	ir	r	r
Input	pr	pr	r	ir	r

for a mapping is determined using the loop relevance principle [31] thereby assuring that a valid mappings for the tuple-list is generated.

6.4.1 Loop Relevance Principle (Proposed by [31]). For each loop in a convolution layer loop-nest the loop relevance principle classifies each of the three operands, input, weight, and output, as being "relevant" (r), "irrelevant" (ir), or "partially relevant" (pr). An operand is classified as irrelevant for a loop if in executing the loop at most one operand element is accessed. An operand is classified as relevant for a loop if in executing the loop all operand elements in the dimension corresponding to the loop are accessed. If only some operand elements are accessed the operand is classified as partially relevant. The interaction of operands in the body of perfectly nested loops determines how operands are classified for each loop. In executing a loop new data elements are accessed for relevant and partially relevant operands and contributes to the memory requirements of the operands. An operand classified as "irrelevant" for a loop require accessing at most one data element of the operand and indicates data reuse opportunities.

Applying the principle to the convolution example in Figure 1 results in an operand classification for the nine loops shown in Table 2. In executing convolution kernel loops all weights are accessed, hence relevant, but only those input elements needed for a convolution operation accessed, hence the input operand is only partially relevant in executing the kernel loops. For each kernel computation only one output element is accessed, thus the Output operand is classified as irrelevant. Similar considerations are used for the other table entries.

For convolution operation with 0-N-dimensional Weights, N-dimensional Input/Output operands, along with one dimension each for Input channels, Output channels, and batch size, the loop relevance relationship between convolution operands and the dimensions are reported in Table 3. Note that since convolution Weights are generally smaller than the Inputs, the Input operand memory requirements depend on size/loop-ranges of the Weight, resulting in a partial relevance for Input operand on Weight and Input/Output dimensions.

Utilizing the loop relevance principle, the potential data reuse of an operand can be computed by taking the product of all its "ir" loops' LPFs from the spatial level (L_S) till a specific hardware level (L_i) (Equation (7)), while the memory required can be computed by taking the product of all of its "r" loops' LPFs (shown in Figure 8 for the three operands). The input operand potential reuse can be computed as MAC operations (computed as the product of all LPFs from spatial level (L_S) till hardware level L_i) divided by the Input operand size at hardware level L_i (Equation (8)).

$$Potential\ data\ reuse\ at\ hardware\ level\ i\ (Weight\ and\ Output) = \prod_{L_S}^{L_i}(LPF_{ir}) \tag{7}$$

$$Potential\ data\ reuse\ at\ hardware\ level\ i\ (Input) = (TotalMACOps_{L_i})/(InputMemoryRequired_{L_i}) \tag{8}$$

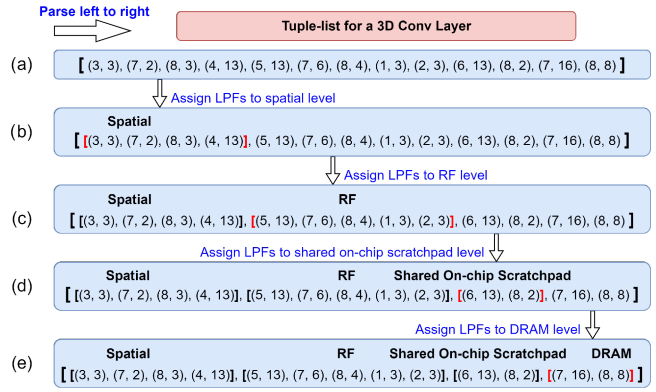


Fig. 7. Steps in generating a valid mapping from a tuple-list for the hardware architecture described in Section 7.1.

6.4.2 Generating a Valid Mapping from Tuple-list. In generating a valid mapping from the tuple-list it is processed in a left-to-right order first assigning one loop iteration per PE. This step terminates when the product of list item LPFs is the maximum not exceeding the number of PEs, thus satisfying the Spatial constraint. This restriction may cause some PEs to not be assigned a loop iteration. No "back-filling" by looking ahead in the tuple list is performed. Potential multi-threading is not covered in the spatial assignment step.

Continuing processing 2-tuples in the tuples-list in a left-to-right order LPFs in the 2-tuples are assigned to each level of each PEs memory hierarchy, one memory level at a time starting with the Register-File followed by assigning list items to the shared memory and terminating with DRAM. For each memory level, the memory for all relevant and partially relevant operands for an LPF assigned to the memory level must result in total memory consumption for the level less than or equal to the memory size at that level: the Memory constraint. Figure 7 gives an example of the generation of a valid mapping and Figure 8 provides pseudocode for the procedure. Once the mapping is determined the potential data reuse for operands up till specific hardware level can be computed using Equation (7).

Figure 7(a) shows a tuple list and Figure 7(b) the result assigning loop iterations to 256 PEs. The spatial scheduling terminates with list item "(4,13)", the last item for which the product of list item LPFs starting with the first does not exceed 256 (the product = $3 \times 2 \times 3 \times 13 = 234$). Next, the maximum number of remaining LPFs taking operand relevance into account are assigned to the PEs RF satisfying the memory constraint for the RF. All PEs for each private memory level are assigned the same amount of distinct data corresponding to indices mapped spatially. For the tuple list in Figure 7, adding the tuples up till "(2,3)" to the RF level results in per PE memory requirement of 216B for Weight, 52B for Output, and 270B for Input and a total memory requirement of 528B satisfying the 672B RF memory constraint (Figure 7(c)). For the shared memory the spatially scheduled LPFs must be accounted for in the memory requirement computation. Adding the LPFs up till "(8,2)", would require 7776B, 40500B, and 52728B for Weight, Input and Output, respectively and a total of 98.63 kiB satisfying the 10 kiB shared memory constraint (Figure 7(d)). The remaining LPFs are similarly assigned to the DRAM (Figure 7(e)). Note that the tile sizes for an operand depend on all its relevant and partially relevant loops at the memory level, but the tile shape and stride depend on which relevant loop IDs contributed to it. Additionally, in this valid mapping representation, the LPFs in each sub-list from left to right define the inner to outer loop order for the specific hardware level.

```
def parse_state_list(state_list):
    # Empty mapping with number of sub-lists equal to the
    # number of hardware levels (1 spatial and rest memory levels)
    # Assume 3 memory levels
    mapping = [[],[],[],[]]
    # Memory required at each memory level
    mem_required = [0, 0, 0]

    #Spatial Constraint
    #******************#
    prod = 1
    for tuple in state_list:
        loopID = tuple(0)
        lpf = tuple(1)

        if lpf * prod <= num_PEs:
            mapping[spatial_level].append(tuple)
            prod *= lpf
        else:
            break
    #Remove tuples assigned to spatial level from state_list
    state_list.remove(mapping[spatial_level])

    #Memory Constraint
    #*****************#
    mem_level = 1   #Innermost memory level (e.g., RF)
    temp_mapping = copy(mapping)

    #Compute memory used so far
    mem_used = compute_mem_req(mapping)
    mem_required = mem_used

    for tuple in state_list:
        loopID = tuple(0)
        lpf = tuple(1)

        TEST:
        temp_mapping[mem_level].append(tuple)
        for operand in ['W','I','O']:
            # Compute memory required if new tuple is added
            if loopID not irrelevant:
                mem_required[mem_level] += compute_mem_req(temp_mapping, operand)

        # If adding new tuple satisfies memory constraint, add tuple to mapping
        if mem_required[mem_level] <= mem_available[mem_level]:
            mapping = temp_mapping
            mem_used = mem_required
        else:
            # Attempt adding new tuple to next memory level
            mem_level += 1
            temp_mapping = mapping
            mem_required = mem_used
            GOTO TEST
```

```
def compute_mem_req(mapping, operand):
    # Compute product of all relevant tuples' LPF of the
    # "operand" based on operand indexing.
    if operand == 'W':
        # Weight
        operand_mem_required =  prod(LPF_R) * prod(LPF_S) * prod(LPF_T)
                              * prod(LPF_C) * prod(LPF_K)
    elif operand == 'O':
        # Output
        operand_mem_required =  prod(LPF_P) * prod(LPF_Q) * prod(LPF_F)
                              * prod(LPF_K) * prod(LPF_B)
    elif operand == 'I':
        # Input
        operand_mem_required =  (prod(LPF_P) + prod(LPF_R) - 1)
                              * (prod(LPF_Q) + prod(LPF_S) - 1)
                              * (prod(LPF_F) + prod(LPF_T) - 1)
                              * prod(LPF_C) * prod(LPF_B)
    return operand_mem_required
```

Fig. 8. Pseudocode for tuple list's conversion to valid mapping.

Comparison to other approaches: In Timeloop the number of factors for each loop matches the number of hardware levels (memory and spatial) and one factor for each loop assigned to a hardware level. Then a check is carried out to determine if the mapping is valid or not. Using this method, we observed in generating 30,720 mappings, valid mappings represented only 4% of all generated mappings. In Chameleon [3], a mapping is reported to be represented using a vector of knobs that include tile sizes for operands and loop unrolling factors, for each of which three possible actions are generated, increment, decrement or stay. Modifying tile sizes using these actions can result in tile sizes being larger than cache sizes. For caches the size mismatch is handled by the cache management mechanism, but for scratchpad memories the mismatch would result in an invalid mapping. Mind Mappings [20] performs gradient updates to the parameters in the mapping representation, that causes parameters like tile sizes, which are integers, to fall out of the integer domain and be invalid. Techniques like Projected Gradient Descent [33] were employed to project any invalid mapping back into the valid domain.

6.4.3 Reward Function. Our reward function is based on the potential data reuse of Weight and Output operands at each memory hierarchy level, except DRAM. There are three main benefits to our reward function.

(1) High data reuse generally is a feature of high-quality (low dynamic energy and high performance) mappings.
(2) Potential operand reuse is simpler and computationally inexpensive to compute compared to [12] used in [3], and we have found to be a good proxy for actual reuse (see Section 7.4.4)
(3) Potential reuse is effective for pruning (see Section 7.4.3), i.e., in our case not performing cost estimation for valid mappings with potential operand reuse below a set level.

Table 4. Hardware Architecture used for Evaluation

DRAM Memory	400 MiB	64b/cycle read/write
Shared Scratchpad Memory (L2)	108 kiB	128b/cycle read/write
Register File (RF)	672B per PE	64b/cycle read/write
MACs	1 per PE	8-bit wide
Number of PEs	256	8-bit data paths

Table 5. 3D Convolution Layers Evaluated by our RL based Mapping Search Heuristic, Loop Ranges, Number of Prime Factors, Search Space Size and the Maximum Potential Data Reuse using Equation (7) for Weight and Output Operands and Computed as (Total Convolution Loop MAC Operations/Input Operand Size) for Input Operand

Layer	Loops						Prime factors	Search Space size	Maximum Potential Data Reuse			Total operations	Maximum Arithmetic Intensity	Layer size (kiB)
	K	C	P	Q	F	R/S/T			Weight	Output	Input			
1	64	28	56	56	56	7	24	6.20E+23	175616	9604	16175.7	2.159E+11	11651.8	18093.0
2	384	256	13	13	13	3	22	1.12E+21	2197	6912	6749.2	1.166E+10	2673.6	4259.6
3	64	3	16	128	171	3	24	6.20E+23	350208	81	1494.9	3.609E+09	152.7	23079.1
4	128	3	16	128	171	3	25	1.55E+25	350208	81	2989.8	7.217E+09	156.7	44972.1
5	256	3	16	128	171	3	26	4.03E+26	350208	81	5979.5	1.443E+10	158.8	88758.2
6	4	256	112	112	112	3	28	3.05E+29	1404928	6912	102.4	7.768E+10	201.8	375901.0
7	256	4	80	80	80	3	28	3.05E+29	512000	108	6418.5	2.818E+10	211.4	130180.8
8	192	64	27	27	27	5	25	1.55E+25	19683	8000	15856.9	6.046E+10	8372.3	7052.5
9	384	192	13	13	13	3	21	5.11E+19	2197	5184	6749.2	8.746E+09	2511.6	3400.7
10	128	4	56	56	56	3	24	6.20E+23	175616	108	3110.7	4.833E+09	207.7	22727.7
11	256	8	28	28	28	3	23	2.59E+22	21952	216	5619.7	2.422E+09	411.2	5752.9
12	32	16	120	120	120	3	27	1.09E+28	1728000	432	822.2	4.772E+10	565.7	82386.1
13	64	64	54	54	54	3	27	1.09E+28	157464	1728	1549.4	3.482E+10	1624.9	20925.5
14	128	64	60	60	60	3	28	3.05E+29	216000	1728	3132.2	9.552E+10	2215.2	42111.5
15	64	3	112	112	112	7	25	1.55E+25	1404928	1029	18770.8	1.850E+11	1948.7	92685.9
16	3	128	32	32	32	3	26	4.03E+26	32768	3456	67.5	6.794E+08	132.2	5019.1
17	192	192	13	13	13	3	20	2.43E+18	2197	5184	3374.6	4.373E+09	2117.5	2016.8
18	256	128	16	16	16	3	30	2.65E+32	4096	3456	4854.5	7.247E+09	2704.2	2617.0
19	64	32	360	360	360	3	32	2.63E+35	46656000	864	1699.5	5.157E+12	1144.9	4398489.3

Also reported are the number of convolution operations, maximum arithmetic intensity and layer size (kiB).

7 EVALUATION

7.1 Hardware Architecture

We evaluated our RL search method for a hardware architecture with 256 PEs each with a 672B (byte) Register File (RF) with 64b/cycle (bits/cycle) read and write bandwidth, a MAC with one cycle latency, no local memory (no L1) and a 108 kiB shared scratchpad memory with 128b/cycle read and write bandwidth as summarized in Table 4. The communication infrastructure is assumed to allow for contention-free communication including multi-and broadcast operations. The read and write bandwidth of the 400 MiB DRAM memory is 64b/cycle. For the specific case of 3D convolution layer #19 (Table 5), a 5 GiB DRAM was defined to fit the layer's operands. Data types are 8 bits [35, 42]. The RF and shared scratchpad memory sizes are similar to Eyeriss [14] which had 512B RF and 108 kiB shared scratchpad memory.

7.2 Mapping Cost Estimation Model and Mapping Quality (EDP) Estimation Parameters

For estimation of mapping energy-delay products (EDPs) we utilize the Timeloop [36] analytical mapping cost estimation model and the Accelergy [43] energy estimation framework coupled with

CACTI 7.0 [10] plugin. The total energy in the Timeloop model is computed as the sum of the energy for each hardware architecture component. For each the energy is computed as the product of the number of operations performed and their respective energy consumption. The read/write energy is specified for each memory level.

We use 28nm CACTI energy estimates. For the shared scratchpad memory, the read and write energy estimations are 5.47 pJ/byte and 4.12 pJ/byte, respectively. For the DRAM memory, both the read and write operations estimates are 512 pJ/byte. We use the 28 nm CMOS and 16b datatype values reported in [44] (which [44] derived using Cadence XtensaProcessor Generator [1]) and scaled to 8b data representation. We use 0.48 pJ/byte for both read and write operations from RF, and 0.0375 pJ/MAC for the PE units. The number of execution cycles for individual hardware components are computed by dividing the workload assigned to the hardware component by its respective capability. For PEs, the number of cycles is simply the number of MAC operations assigned to it. For the memory modules, the number of cycles is computed by dividing data volume accessed (bytes) by the memory width (bytes/cycle). The execution cycles for a mapping assumes fully overlapped communication and computation and hence is the maximum cycles required by a hardware component.

7.3 Test Case

7.3.1 3D Convolution Layers. We evaluate our RL search heuristic in finding mappings of 19 3D convolution layers (Table 5) onto the hardware architecture in Section 7.1. Table 5 shows the number of prime factors, the search space size (factorial of number of prime factors), the "Total operations" computed as "(2*Total convolution layer MAC operations) – (Number of Output operand elements)" since there is one less addition than multiplication in a convolution computation. The maximum arithmetic intensity (Ops/byte) is computed as "Total operations/layer size in bytes". The layer size is computed as the sum of the Input, Weight, and Output operands' memory requirements, multiplied by the data precision (8 bits used for our experiments). The memory requirements for each operand is computed as shown in Figure 8. The arithmetic intensity with respect to DRAM is computed as "Total operations/Total bytes accessed from DRAM". A batch size of one was used in our experiments.

7.3.2 RL Search Implementation and Hyperparameters. Our experiments were performed on an Intel Xeon G6252 24 cores, 2.1GHz, 189GB DDR4 memory University of Houston's Carya cluster node. The RL search heuristic is implemented in TensorFlow 2.9.1 [2] and Python 3.9.7. For RL search result reproducibility, we fixed the random number seeds for Python runtime, NumPy [18] and TensorFlow packages. The RL agent hyperparameters and RL search parameters are reported in Table 6. We run our RL based search heuristic for 30 episodes with each episode performing 1024 "tuple-list – action – reward" events. Thus, each RL search visits 30,720 states. The reward function parameters reported were chosen based on experimentation. We use single precision weights for the Actor-Critic neural networks. For the convolution layer with most LPFs, layer 19 with 32 LPFs (see Table 5) the Actor-Critic networks memory requirements were 11.56 kiB and 4.14 kiB respectively.

7.3.3 Evaluation Metrics. The evaluation metrics reported below are: (1) the search computational requirement estimated from floating-point instructions executed, and (2) the mapping EDP. The performance counter tool LIKWID [41] (likwid-perfctr) used to acquire floating-point instruction counts reported (1) FP_ARITH_INST_RETIRED_128B_PACKED_DOUBLE: 128-bit packed double precision counts (two double precision operations per instruction). (2) FP_ARITH_INST_RETIRED_SCALAR_DOUBLE: Count of Scalar operations on double precision values (one double

Table 6. RL Hyperparameters, Search and Reward Function Parameters

RL Hyperparameters	
Optimizer used for learning	Adam
Learning rate	1.0×10^{-3}
Discount factor for PPO	0.95
Generalized Advantage Estimation (GAE) for PPO	0.99
Policy clipping parameter	0.2
RL Search Parameters	
Episodes ($N_{episodes}$)	30
Total trajectory length ($N_{totalTrajectory}$)	1024
Learning trajectory length ($N_{learningTrajectory}$)	512
Batch Size ($N_{batchSize}$)	256
Training epochs (N_{epochs})	3
Reward Function Parameters	
norm_factor	4096
alpha	4.321
clipping factor	25

precision operation per instruction). The total number of double precision operations is computed as "(packed_double * 2) + (scalar_double)".

7.4 RL Characterization

7.4.1 Initial Tuple-lists. We evaluated our RL method for the 10 different initial tuple lists below for each convolution layer:

(1) LPFs of loops generated in order from left to right: K,C,F,Q,P,T,S,R (initial list ID: 1)
(2) LPFs of loops generated in order from left to right: R,S,T,P,Q,F,C,K (initial list ID: 2)
(3) LPFs of loops generated in order from left to right: K,C,F,Q,P,T,S,R, then sorted in ascending order based on the LPF value (initial list ID: 3)
(4) LPFs of loops generated in order from left to right: R,S,T,P,Q,F,C,K, then sorted in ascending order based on the LPF value (initial list ID: 4)
(5) LPFs of loops generated in order from left to right: R,S,T,P,Q,F,K,C (initial list ID: 5)
(6) LPFs of loops generated in order from left to right: R,S,T,P,Q,K,F,C (initial list ID: 6)
(7) LPFs of loops generated in order from left to right: R,S,T,P,K,Q,F,C (initial list ID: 7)
(8) LPFs of loops generated in order from left to right: R,S,T,P,Q,C,F,K (initial list ID: 8)
(9) LPFs of loops generated in order from left to right: R,S,T,P,C,Q,F, K (initial list ID: 9)
(10) LPFs of loops generated in order from left to right: R,S,T,C,P,Q,F,K (initial list ID: 10)

7.4.2 RL Method Computational Breakdown. Figure 9 reports the breakdown and ranges of the number of operations for Actor-Critic inference (using Equation (3) and 4), Actor-Critic training (using Equation (2)) and mapping evaluations for the 10 RL search initial tuple-lists. The inference contributed 5.11% and the training 15.46% on average to the total operations for the 19 layers and the 10 initial tuple-lists. On average 79.42% of operations were required for mapping evaluations.

7.4.3 Pruning Impact on Number of Valid Mappings Evaluated. A user defined threshold for the Weight and Output operand reuse is used not to evaluate the quality of valid mappings with potential reuse below the threshold. Since mapping cost evaluation is computationally demanding, see Figure 9, avoiding evaluation of mappings unlikely to result in high quality mappings is essential

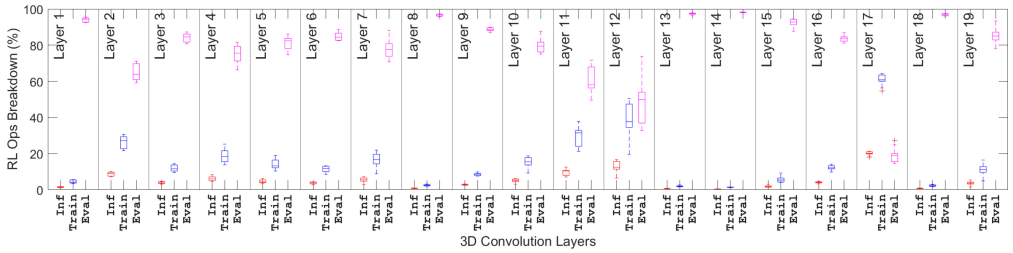
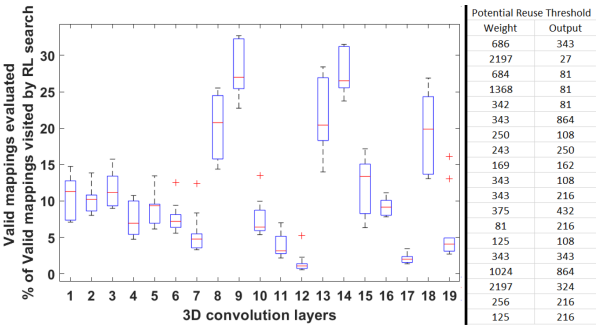


Fig. 9. Box plot showing breakdown of the number of operations for Actor-Critic inference (Inf), Training (Train) and mapping evaluations (Eval) for the 10 initial tuple-list used for our RL search.



Potential Reuse Threshold	
Weight	Output
686	343
2197	27
684	81
1368	81
342	81
343	864
250	108
243	250
169	162
343	108
343	216
375	432
81	216
125	108
343	343
1024	864
2197	324
256	216
125	216

Fig. 10. The percentage of 30,720 valid mappings evaluated for each of the 3D convolution layers, and the potential reuse threshold used for Weight and Output operands.

for a computationally efficient search of high-quality mappings. Figure 10 shows a box plot of fraction of the 30,720 mappings for each of the 19 convolution layers for the 10 initial tuple-lists that were evaluated for cost with pruning level reported in Figure 10. The highest average percentage of valid mappings evaluated for the 10 initial tuple-lists was 27.7% for layer #14 and the minimum average was 1.5% for layer #12. Though the pruning based on potential data reuse threshold may miss potentially high quality mappings, our results demonstrate that the EDP of evaluated mappings with few exceptions are better than those achieved by Timeloop (Section 7.5.2) Future work includes developing a model for choosing the reuse threshold based on the convolution loop characteristics and hardware architecture features.

7.4.4 Justification and Validation of our Reward Function. **Justification:** For the convolution layers reported in Table 5, our reward function uses the potential data reuse of Weight and Output operands at the spatial and RF memory levels, Figure 11. The pruning threshold for valid mappings is also based on the potential reuse achieved at the spatial and RF levels. Figure 12 (bar graph, left y-axis) shows the energy breakdown between the MAC units, and the RF, shared memory, and DRAM memory levels for the lowest (marked as 'L') and highest (marked as 'H') EDP mappings for each of the 19 convolution layers of the 30,720 mappings for the initial tuple-list numbers shown at Figure 12 (bottom), and defined in Section 7.4.1. The left energy axis is shown on a log10 scale. The arithmetic intensity computed as "2*Total convolution layer MAC instructions/sum of DRAM memory access for the three operands" is shown as a line with the scale on the right. A high arithmetic intensity indicates low memory accesses from the DRAM and high reuse of the operands in the RF and shared memory combined. The box labels report the % of maximum arithmetic intensity (see Table 3) exploited. There is a high correlation of the energy consumed

```
def reward_func():
    if (weight_potential_reuse_spatial * weight_potential_reuse_RF) < weight_potential_reuse_threshold:
        reward = 0
        return reward
    elif (output_potential_reuse_spatial * output_potential_reuse_RF) < output_potential_reuse_threshold:
        reward = 0
        return reward
    else:
        reward = ((weight_potential_reuse_spatial * weight_potential_reuse_RF)/norm_factor + 1)^alpha +
                 ((output_potential_reuse_spatial * output_potential_reuse_RF)/norm_factor + 1)^alpha

        reward = clip(reward, min=0, max=clipping_factor)
        return reward
```

Fig. 11. Reward function for our RL based mapping search heuristic.

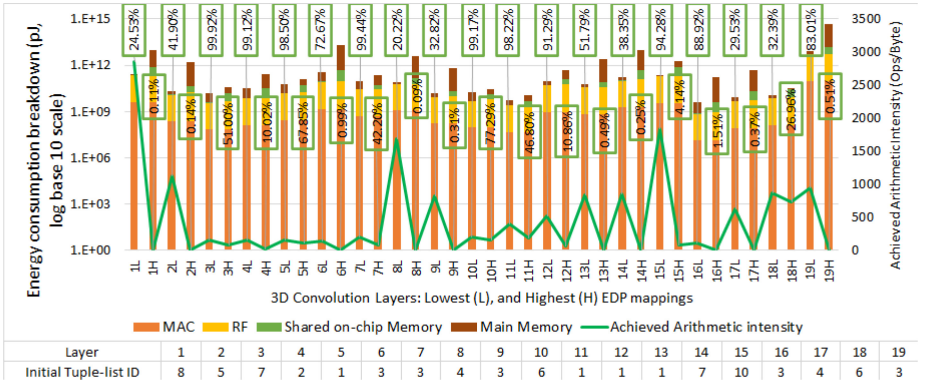


Fig. 12. Energy consumption breakdown for the lowest (L) and highest (H) EDP mappings found using our RL search for the 19 3D convolution layers (bar graph) evaluated using the initial tuple-list IDs shown at bottom. Achieved arithmetic intensity (ops/byte) shown by the line graph. The box labels show the % of maximum arithmetic intensity exploited for each layer.

at the RF and DRAM levels with the achieved arithmetic intensity. For the mappings with high absolute achieved arithmetic intensity (or high % of maximum arithmetic intensity), the energy is dominated by the RF due to majority of data accesses being performed from it. For mappings with low absolute arithmetic intensity (or low % of maximum arithmetic intensity) the operand reads from DRAM were significantly higher than for high arithmetic intensity mappings, thus resulting in high DRAM energy, and consequently high total energy. We conclude the use of potential operand reuse as a reward function was effective in finding good EDP mappings.

Validation: To assess whether potential data reuse is a good proxy for actual data use we report the actual reuse as percent of the potential data reuse at the shared memory level for Weight and Output operands for the best EDP mapping of 30,720 valid mappings (30 episodes each of 1024 states) for each of 19 convolution layers for all 10 initial tuple-lists (Figure 13). The actual shared memory accesses are the sum of operand RF loads and stores and loads from DRAM. The actual data reuse at the scratchpad level for Weights and Output operands is computed as "(Total convolution layer MAC instructions)/(operand accesses to shared memory)". Figure 13 shows a histogram of 190 mappings (lowest EDP mappings found for each of the 19 3D convolution layers using the 10 initial tuple-lists in Section 7.4.1) binned based on the % of potential data reuse exploited. For the Weight operand, 85 mappings exploited 95–100%, three mappings 90–95%, five mappings 85–90%, one mapping 75%, one mapping 74.91%, five mappings 70–75%, eight mappings 65–70%, and 83 mapping exploited 45–50% of the potential reuse. For the Output operand, 158 mappings exploited 95–100%, eight mappings 90–95%, two mappings 85–90%, one mapping 82%,

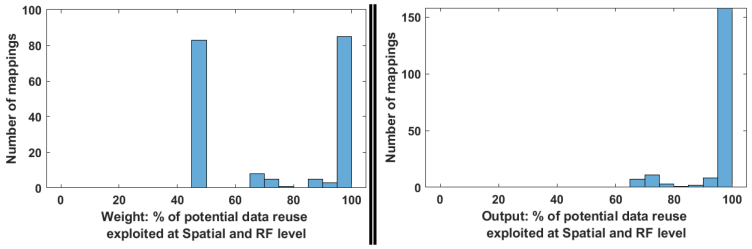


Fig. 13. Histogram with mappings binned based on the % of potential data reuse at spatial and RF level combined exploited. Number of mappings = 190, number of bins = 20, bin width = 5, bin limits = [0, 100].

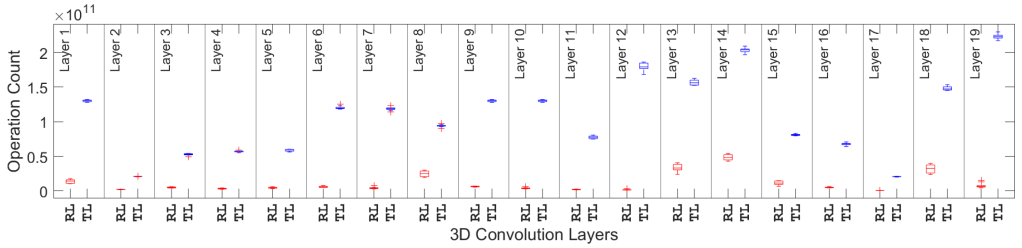


Fig. 14. Box plot shows for each 3D convolution layer the total operation count for the RL search (RL, red) for each of the 10 initial tuple-list used and Timeloop random search's (TL, blue) 10 random seed trials. RL operation count includes operations for Actor-Critic inference and training, and mapping evaluations.

three mappings 75–80%, 11 mappings 70–75%, and seven mappings 65–70%. These correlations demonstrate the effectiveness of using potential data reuse as a proxy for actual data reuse in the reward function for our mapping search heuristic.

7.5 RL and Timeloop Comparison

Both our RL search and Timeloop have "randomness" built in. For our RL method it stems from the choice of initial tuple-list and the softmax function used for determining tuple-list swap actions. For Timeloop potential mappings are chosen randomly with only about 4% being valid mappings. To assess the impact of the randomness we used 10 trials for each method. For the RL search, the 10 different initial tuple-list used were listed in Section 7.4.1. For Timeloop random number seeds 1 through 10 was used for the "minstd_rand0" [7] random number generator. Each search was terminated when 30,720 valid mappings had been found. Between the 10 Timeloop trials for each layer, we observed little to no overlap between the valid mappings found. Thus, the basis for comparison with our RL search is approximately 307,200 Timeloop valid mappings for each convolution layer.

7.5.1 Computational Effort Comparison for Assessing Same Number of Valid Mappings (30,720). Figure 14 shows, for each 3D convolution layer, the number of operations performed by our RL search for the 10 initial tuple-lists (RL, red), and the Timeloop operations for 10 random number seeds. Our RL method for the 19 layers performs only an average of 11.24% of the operations of that of Timeloop, with a maximum average of 29.17% for layer 9 (across the 10 initial tuple-list) and a minimum average of 1.07% for layer 12 (across the 10 initial tuple-list). This difference is mainly because Timeloop evaluates all 30,720 valid mappings, whereas our pruning strategy significantly reduces the computationally expensive mapping cost estimation. Further our reward function is computationally inexpensive, and as seen in the next section the EDP for the RL found mappings

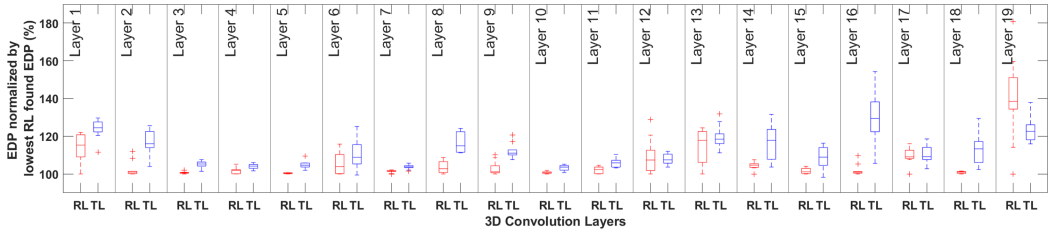


Fig. 15. Box plot showing for each 3D convolution layer, the lowest EDP found using 10 initial tuple-list with the RL search (Red, 'RL') and with 10 trials of Timeloop's random search (blue, 'TL'), normalized by the lowest EDP mapping found with RL across the 10 initial tuple-list.

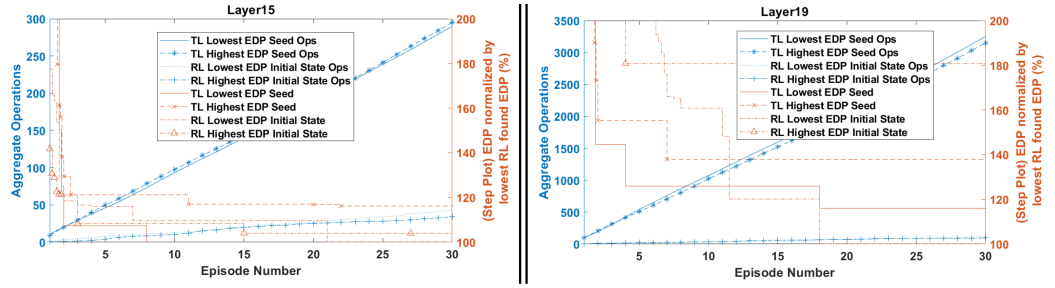


Fig. 16. X-axis shows the RL episode number (each episode is 1024 state transitions). Line plot (left y-axis) shows the aggregate operations of Timeloop's (TL) random number seed and RL's (RL) initial tuple-list resulting in lowest and highest EDP mappings normalized by total operations performed in first episode of RL's lowest EDP mapping case. Step plot (right y-axis) shows the execution trace of the 4 cases. The values plotted on right y-axis are EDP normalized by the lowest EDP found using RL.

with a few exceptions lower than that of Timeloop. The search space sizes for the layers varied between 2.4×10^{18} and 2.6×10^{35}.

7.5.2 Mapping Quality Comparison. The box plot (Figure 15) shows for each 3D convolution layer, the lowest EDP of evaluated mappings for 307,200 valid mappings visited (10 initial tuple-list, 30 epochs of 1024 mappings each) using our RL based search heuristic (red, RL), and the lowest EDP for the same number of valid mappings found by Timeloop's random search, normalized by the lowest RL EDP mapping. On average for the 19 layers the lowest EDP found by Timeloop were 12.51% higher than the lowest EDP found by our RL search. The best-case comparison for our RL search was for layer 19 (random seed 4) for which the lowest EDP for Timeloop was 15.95% higher than the RL search lowest EDP. The worst-case comparison for our RL search was for layers 6 and 15. For layer 6 (random seed 3) the lowest EDP for Timeloop was 0.6% lower than the RL search lowest EDP and for layer 15 (random seed 4) the lowest EDP for Timeloop was 1.8% lower than the lowest EDP found by RL search.

7.5.3 Mapping Quality as a Function of Computational Effort. Figure 16 (line plot, left y-axis) shows the aggregate operations of Timeloop's (TL) random number seed and RL's (RL) initial tuple-list resulting in lowest and highest EDP mappings normalized by total operations performed in first episode of RL's lowest EDP mapping case with the search progression for two convolution layers at a granularity of 1024 valid mappings (one RL episode). For layer 15 the lowest EDP mapping found by Timeloop for 30,720 valid mappings was 1.8% lower than the Lowest EDP mapping found by our RL method, the worst relative performance for the RL method. For layer 19 the difference

in the lowest EDP mapping of 30,720 valid mappings was the largest in the RL methods favor with Timeloop's lowest EDP being 15.9% higher. For both methods the trials (Timeloop random number seeds, RL initial tuple lists) resulting in the lowest and highest EDP mappings are shown. For the RL method the aggregate operations include operations for Actor-Critic inference and training, and environment mapping evaluation. For layer 15 and 19 (Figure 16), Timeloop performs an estimated 7.7× and 32.8× higher operations per RL episode. The operations per RL episode are estimated by performing a linear least-squares fit to the number of aggregate operations for both the RL and Timeloop methods as a function of number of episodes. Across the 19 convolution layers evaluated and the 10 initial tuple-lists used for the RL method, Timeloop's 10 trials each require an average 21.3× higher operations per RL episode, with a minimum of 3.8× for layer 8 and maximum of 106× for layer 12. Figure 16 step-plot (right y-axis) shows the corresponding EDP evolution with the EDPs normalized by the lowest EDP found using the RL method.

7.5.4 Comparison to Lower Bound. We also compared the lowest EDP mapping found for each 3D convolution layer to the theoretical lower bound EDP for each layer. We computed the lower bound EDP as "Minimum execution cycles * Minimum energy consumption". The minimum execution cycles were computed by dividing the number of convolution layer operations by the peak theoretical performance for the PEs, i.e., 2 ops/cycle/PE for 256 PEs. For the lowest EDP mappings found using our RL search, the number of execution cycles for convolution operations were larger than the execution cycles for memory operations. The minimum energy consumption for arithmetic operations was computed as total number of MAC operations multiplied by energy per MAC operation. For memory the energy was similarly determined as the product of total number of read and write operations and the energy for each such operation at the RF, shared memory, and DRAM levels. For the Output operand at the DRAM level only write energy is considered in the lower bound estimate. For the 19 layers the mappings with lowest EDP had on average 4.69× higher EDP than the lower bound EDP, with a minimum of 1.29× for layer 3 and a maximum of 14.59× for layer 1. Out of 19 3D convolution layers, the lowest EDP mappings did not achieve 100% PE utilization for only 3 layers. The reason therefore was that the product of LPFs at the spatial level was smaller than the number of PEs (256).

8 CONCLUSION

In this work, we demonstrated a computationally efficient DNN mapping search heuristic using Deep Reinforcement Learning. Our novel RL reward function based on the potential data reuse and the mapping pruning strategy based on the potential data reuse threshold of the operands together resulted in only requiring an average 11.24% of computational requirements of Timeloop's random search across 19 3D convolution layers evaluated for assessing same number of mappings. The computational cost savings are also demonstrated by the fact that Timeloop required an average 21.3× higher operations per RL episode (1024 state transitions). The mappings found by Timeloop were an average 12.51% higher than the lowest EDP mappings found by our RL search, with the lowest EDP mappings found using our RL search being an average 4.69× higher than the lower bound EDP, with the best case being only 1.29× higher. Further, our novel state representation (tuple-list) along with the parsing strategy ensures that only valid mappings are generated, while with Timeloop, only 4% of mappings visited were valid mappings.

REFERENCES

[1] 2023. Cadence. https://www.cadence.com/en_US/home/tools/ip/tensilica-ip.html

[2] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, et al. 2016. TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems. https://doi.org/10.48550/ARXIV.1603.04467

[3] Byung Hoon Ahn, Prannoy Pilligundla, Amir Yazdanbakhsh, and Hadi Esmaeilzadeh. 2020. Chameleon: Adaptive code optimization for expedited deep neural network compilation. *CoRR* abs/2001.08743 (2020). arXiv:2001.08743 https://arxiv.org/abs/2001.08743
[4] Murtaza Ali, Eric Stotzer, Francisco D. Igual, and Robert A. van de Geijn. 2012. Level-3 BLAS on the TI C6678 multi-core DSP. In *2012 IEEE 24th International Symposium on Computer Architecture and High Performance Computing*. 179–186. https://doi.org/10.1109/SBAC-PAD.2012.26
[5] Anon. 2021. F3 Netherlands" Marine Seismic Dataset. https://terranubis.com/datainfo/F3-Demo-2020
[6] Anon. 2023. Intel oneAPI Math Kernel Library. https://www.intel.com/content/www/us/en/develop/documentation/onemkl-developer-reference-c/top.html
[7] Anon. 2023. Minimal Standard Minstd_rand0 Generator. https://cplusplus.com/reference/random/minstd_rand0/
[8] Anon. 2023. NVIDIA CUDA Basic Linear Algebra Subroutine Library. https://docs.nvidia.com/cuda/cublas/
[9] Suyash Bakshi and Lennart Johnsson. 2020. A highly efficient SGEMM implementation using DMA on the Intel/Movidius Myriad-2. In *2020 IEEE 32nd International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD'20)*. 321–328. https://doi.org/10.1109/SBAC-PAD49847.2020.00051
[10] Rajeev Balasubramonian, Andrew B. Kahng, Naveen Muralimanohar, Ali Shafiee, and Vaishnav Srinivas. 2017. CACTI 7: New tools for interconnect exploration in innovative off-chip memories. *ACM Trans. Archit. Code Optim.* 14, 2, Article 14 (june 2017), 25 pages. https://doi.org/10.1145/3085572
[11] Brendan Barry, Cormac Brick, Fergal Connor, David Donohoe, David Moloney, et al. 2015. Always-on vision processing unit for mobile applications. *IEEE Micro* 35, 2 (2015), 56–66. https://doi.org/10.1109/MM.2015.10
[12] Tianqi Chen and Carlos Guestrin. 2016. XGBoost. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM. https://doi.org/10.1145/2939672.2939785
[13] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, et al. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. https://doi.org/10.48550/ARXIV.1802.04799
[14] Yu-Hsin Chen, Joel Emer, and Vivienne Sze. 2016. Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA'16)*. 367–379. https://doi.org/10.1109/ISCA.2016.40
[15] R. Clint Whaley, Antoine Petitet, and Jack J. Dongarra. 2001. Automated empirical optimizations of software and the ATLAS project. *Parallel Comput.* 27, 1 (2001), 3–35. https://doi.org/10.1016/S0167-8191(00)00087-9 New Trends in High Performance Computing.
[16] William J. Dally, Yatish Turakhia, and Song Han. 2020. Domain-specific hardware accelerators. *Commun. ACM* 63, 7 (June 2020), 48–57. https://doi.org/10.1145/3361682
[17] Shail Dave, Youngbin Kim, Sasikanth Avancha, Kyoungwoo Lee, and Aviral Shrivastava. 2019. DMazeRunner: Executing perfectly nested loops on dataflow accelerators. *ACM Trans. Embed. Comput. Syst.* 18, 5s, Article 70 (oct 2019), 27 pages. https://doi.org/10.1145/3358198
[18] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, et al. 2020. Array programming with NumPy. *Nature* 585, 7825 (Sept. 2020), 357–362. https://doi.org/10.1038/s41586-020-2649-2
[19] Kartik Hegde, Rohit Agrawal, Yulun Yao, and Christopher W. Fletcher. 2018. Morph: Flexible acceleration for 3D CNN-based video understanding. *CoRR* abs/1810.06807 (2018). arXiv:1810.06807 http://arxiv.org/abs/1810.06807
[20] Kartik Hegde, Po-An Tsai, Sitao Huang, Vikas Chandra, Angshuman Parashar, et al. 2021. Mind mappings: Enabling efficient algorithm-accelerator mapping space search. *CoRR* abs/2103.01489 (2021). arXiv:2103.01489 https://arxiv.org/abs/2103.01489
[21] Marius Hobbhahn. 2021. How to Measure FLOP/s for Neural Networks Empirically?
[22] Qijing Huang, Minwoo Kang, Grace Dinh, Thomas Norell, Aravind Kalaiah, et al. 2021. CoSA: Scheduling by Constrained Optimization for Spatial Accelerators. https://doi.org/10.48550/ARXIV.2105.01898
[23] Hong Jia-Wei and H. T. Kung. 1981. I/O complexity: The red-blue pebble game. In *Proceedings of the Thirteenth Annual ACM Symposium on Theory of Computing (STOC'81)*. Association for Computing Machinery, New York, NY, USA, 326–333. https://doi.org/10.1145/800076.802486
[24] Norman P. Jouppi, Cliff Young, Nishant Patil, David A. Patterson, Gaurav Agrawal, et al. 2017. In-datacenter performance analysis of a tensor processing unit. *CoRR* abs/1704.04760 (2017). arXiv:1704.04760 http://arxiv.org/abs/1704.04760
[25] Sheng-Chun Kao and Tushar Krishna. 2020. GAMMA: Automating the HW mapping of DNN models on accelerators via genetic algorithm. In *Proceedings of the 39th International Conference on Computer-Aided Design (ICCAD'20)*. Association for Computing Machinery, New York, NY, USA, Article 44, 9 pages. https://doi.org/10.1145/3400302.3415639
[26] Induprakas Kodukula, Nawaaz Ahmed, and d Keshav Pingali. 1997. Data-centric multi-level blocking. 346–357.
[27] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2017. ImageNet classification with deep convolutional neural networks. *Commun. ACM* 60, 6 (may 2017), 84–90. https://doi.org/10.1145/3065386

[28] Jakub Kurzak, Wesley Alvaro, and Jack Dongarra. 2009. Optimizing matrix multiplication for a short-vector SIMD architecture – CELL processor. *Parallel Comput.* 35 (03 2009), 138–150. https://doi.org/10.1016/j.parco.2008.12.010

[29] Hyoukjun Kwon, Prasanth Chatarasi, Michael Pellauer, Angshuman Parashar, Vivek Sarkar, et al. 2018. Understanding reuse, performance, and hardware cost of DNN dataflows: A data-centric approach using MAESTRO. https://doi.org/10.48550/ARXIV.1805.02566

[30] Michael Laskin, Luke Metz, Seth Nabarro, Mark Saroufim, Badreddine Noune, et al. 2020. Parallel training of deep networks with local updates. https://doi.org/10.48550/ARXIV.2012.03837

[31] Linyan Mei, Pouya Houshmand, Vikram Jain, Sebastian Giraldo, and Marian Verhelst. 2021. ZigZag: Enlarging joint architecture-mapping design space exploration for DNN accelerators. *IEEE Trans. Comput.* 70, 8 (2021), 1160–1174. https://doi.org/10.1109/TC.2021.3059962

[32] John Mellor-Crummey, David Whalley, and Ken Kennedy. 2001. Improving memory hierarchy performance for irregular applications using data and computation reorderings. *International Journal of Parallel Programming* 29 (01 2001). https://doi.org/10.1023/A:1011119519789

[33] Yurii Nesterov. 2014. *Introductory Lectures on Convex Optimization: A Basic Course* (1st ed.). Springer Publishing Company, Incorporated.

[34] Gilbert Netzer. 2015. *Efficient LU Factorization for Texas Instruments Keystone Architecture Digital Signal Processors.* Master's Thesis. Royal Institute of Technology (KTH). http://www.diva-portal.org/smash/get/diva2:837145/FULLTEXT01

[35] Badreddine Noune, Philip Jones, Daniel Justus, Dominic Masters, and Carlo Luschi. 2022. 8-bit Numerical Formats for Deep Neural Networks. arXiv:cs.LG/2206.02915

[36] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A. Ying, et al. 2019. Timeloop: A systematic approach to DNN accelerator evaluation. In *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS'19).* 304–315. https://doi.org/10.1109/ISPASS.2019.00042

[37] Fabian Schuiki, Michael Schaffner, and Luca Benini. 2019. NTX: An energy-efficient streaming accelerator for floating-point generalized reduction workloads in 22 nm FD-SOI. In *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE'19).* 662–667. https://doi.org/10.23919/DATE.2019.8715007

[38] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. https://doi.org/10.48550/ARXIV.1707.06347

[39] Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. https://doi.org/10.48550/ARXIV.1409.1556

[40] Du Tran, Lubomir Bourdev, Rob Fergus, Lorenzo Torresani, and Manohar Paluri. 2014. Learning spatiotemporal features with 3D convolutional networks. https://doi.org/10.48550/ARXIV.1412.0767

[41] Jan Treibig, Georg Hager, and Gerhard Wellein. 2010. LIKWID: A lightweight performance-oriented tool suite for x86 multicore environments. In *2010 39th International Conference on Parallel Processing Workshops.* 207–216. https://doi.org/10.1109/ICPPW.2010.38

[42] Naigang Wang, Jungwook Choi, Daniel Brand, Chia-Yu Chen, and Kailash Gopalakrishnan. 2018. Training deep neural networks with 8-bit floating point numbers. arXiv:cs.LG/1812.08011

[43] Yannan Nellie Wu, Joel S. Emer, and Vivienne Sze. 2019. Accelergy: An architecture-level energy estimation methodology for accelerator designs. *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)* (2019), 1–8.

[44] Xuan Yang, Mingyu Gao, Qiaoyi Liu, Jeff Setter, Jing Pu, et al. 2020. Interstellar: Using halide's scheduling language to analyze DNN accelerators. 369–383. https://doi.org/10.1145/3373376.3378514

[45] Tim Zerrell and Jeremy Bruestle. 2019. Stripe: Tensor compilation via the nested polyhedral model. https://doi.org/10.48550/ARXIV.1903.06498

[46] Size Zheng, Yun Liang, Shuo Wang, Renze Chen, and Kaiwen Sheng. 2020. FlexTensor: An automatic schedule exploration and optimization framework for tensor computation on heterogeneous system. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'20).* Association for Computing Machinery, New York, NY, USA, 859–873. https://doi.org/10.1145/3373376.3378508

[47] Özgün Çiçek, Ahmed Abdulkadir, Soeren S. Lienkamp, Thomas Brox, and Olaf Ronneberger. 2016. 3D U-Net: Learning dense volumetric segmentation from sparse annotation. https://doi.org/10.48550/ARXIV.1606.06650

Received 16 March 2023; revised 1 June 2023; accepted 13 July 2023